FAQ Generation Pipeline

Stage-by-stage design document for the medical FAQ pipeline.

1. Overview

Eight stages, all extractive and rule-based. No LLM is invoked at any point. The pipeline pulls chat sessions from MongoDB, prepares and embeds the questions, clusters them, picks a canonical answer from real agent replies, polishes the wording, and writes the FAQ back to MongoDB for clinician review.

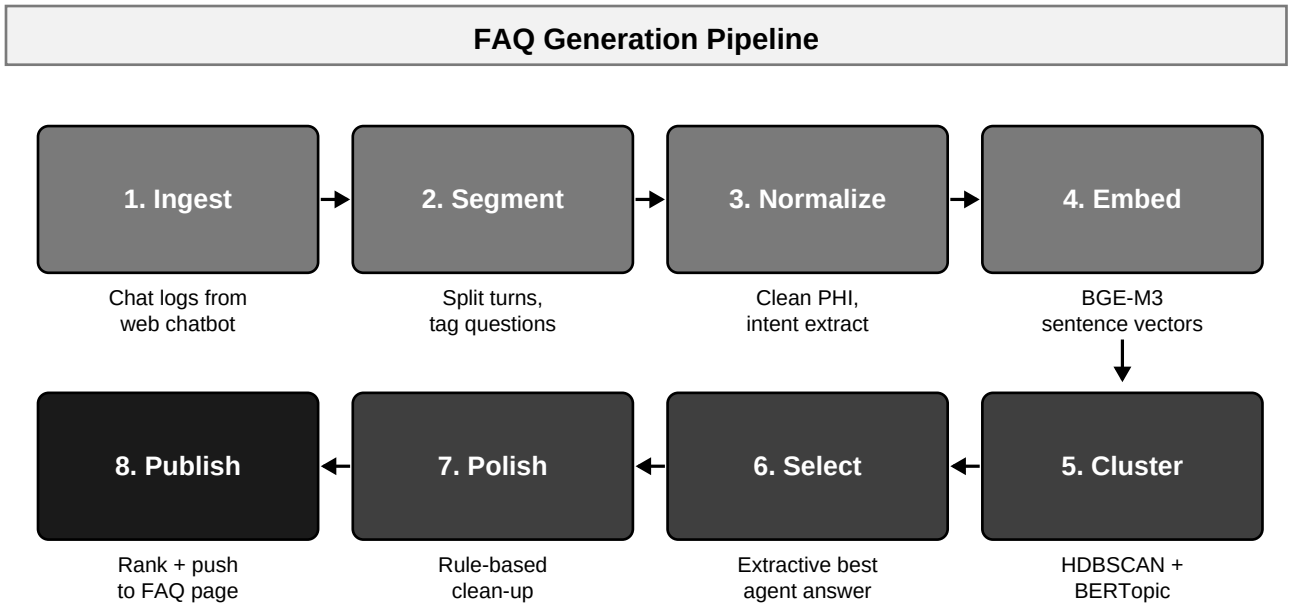
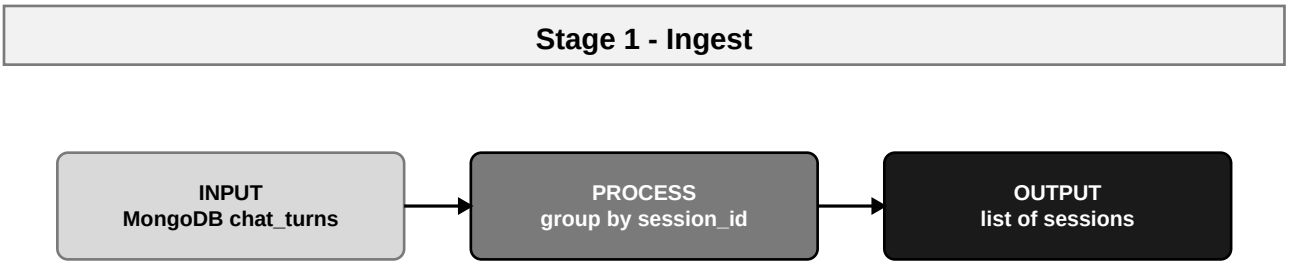


Fig 1. End-to-end pipeline.

2. Stages 1 to 4 - Preparation

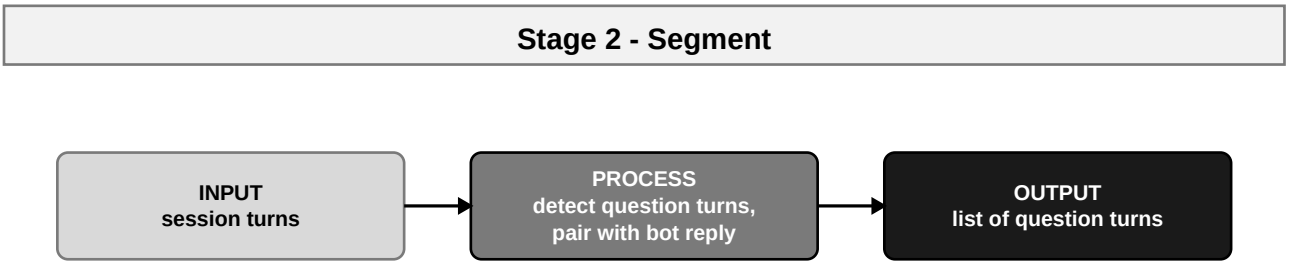
Stage 1: Ingest

Pulls every session from MongoDB written since the last run. Groups turns by session_id. Optionally loads partner-clinic export files.



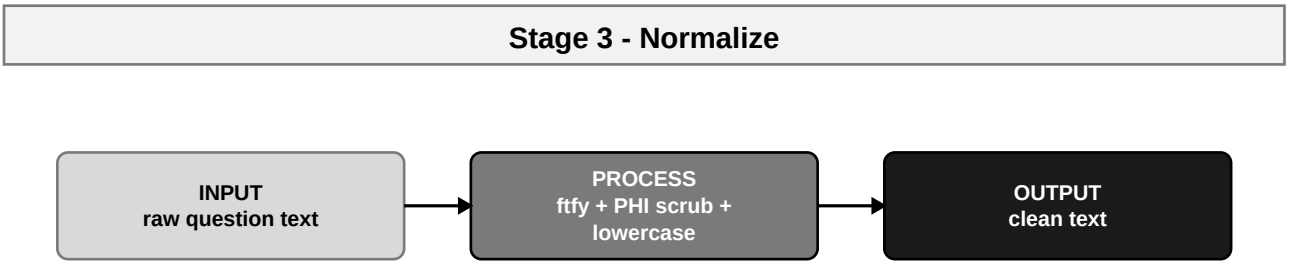
Stage 2: Segment

Detects user questions (punctuation, question starter words, including Urdu-roman starters). Pairs each question with the bot reply that followed it; that pairing becomes the candidate answer.



Stage 3: Normalize

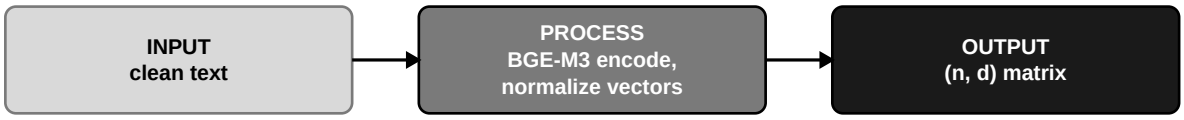
Strips PHI (IDs, phone numbers, emails, dates). Fixes encoding issues with ftfy. Lower-cases and squashes whitespace.



Stage 4: Embed

Encodes each clean question with BGE-M3, normalised vectors. Result is an (n, d) NumPy matrix aligned with the turn list.

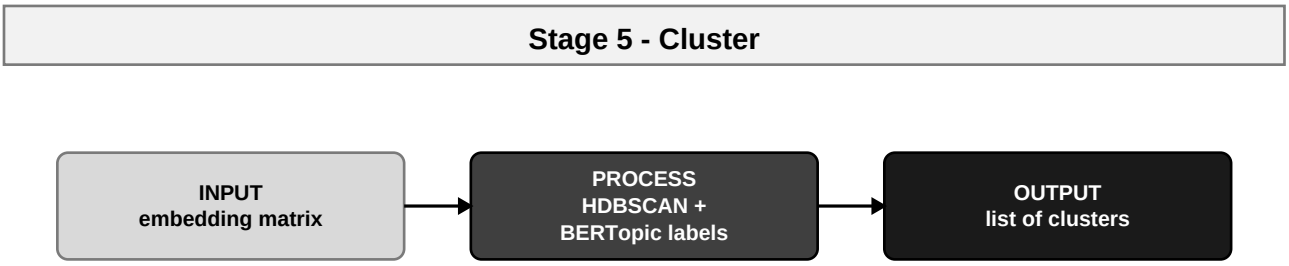
Stage 4 - Embed



3. Stages 5 to 8 - Selection and Publish

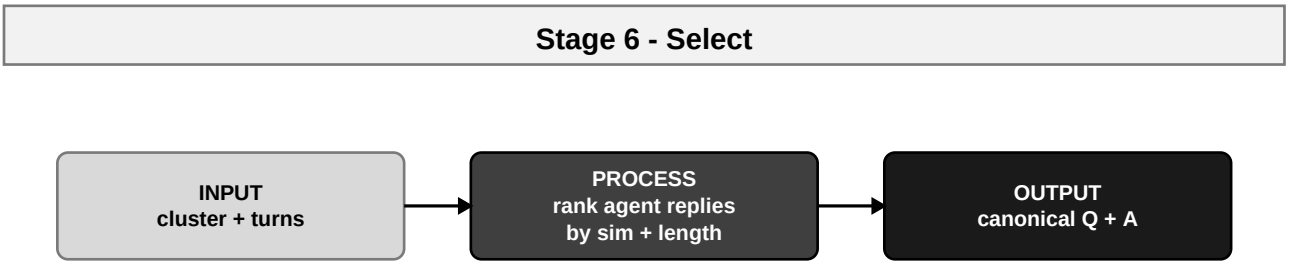
Stage 5: Cluster

HDBSCAN with min_cluster_size=5 on euclidean-normalised vectors. Noise points (label -1) are dropped. BERTopic provides a topic label per cluster for the admin dashboard.



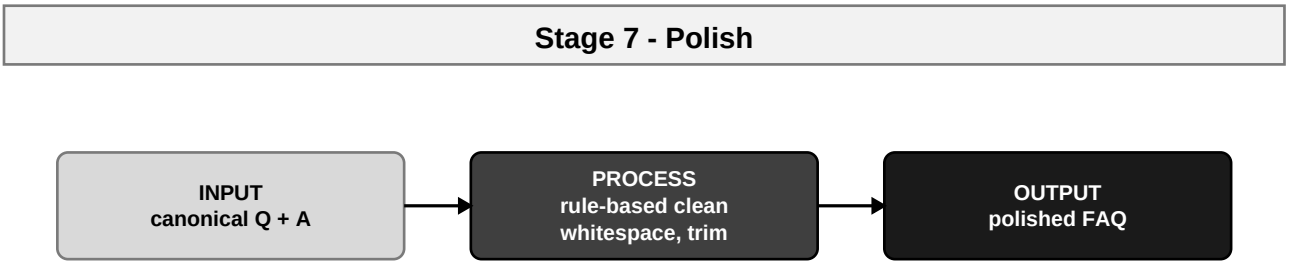
Stage 6: Select

Extractive. For each cluster, rank candidate agent replies by centroid similarity, length, and recency. Pick the top one as the canonical answer. Fall back to the longest non-empty reply if the top scoring reply is empty.



Stage 7: Polish

Rule-based clean-up only. Normalise whitespace, fix punctuation, trim to a configured maximum sentence count, optional greeting and closing prefix.



Stage 8: Publish

Upserts each FAQ to mongo.faqs keyed on (specialty, cluster_id) with approved=false. Admin dashboard reviews and approves.

Stage 8 - Publish

4. Cluster Visualisation

After Stage 4 the embeddings sit in a 1024-dimensional space. For the admin dashboard and reports, we project them to two dimensions with UMAP. Each cluster shows up as a coloured blob. Noise points are dropped before the projection.

Embedding space (2D UMAP) - mock

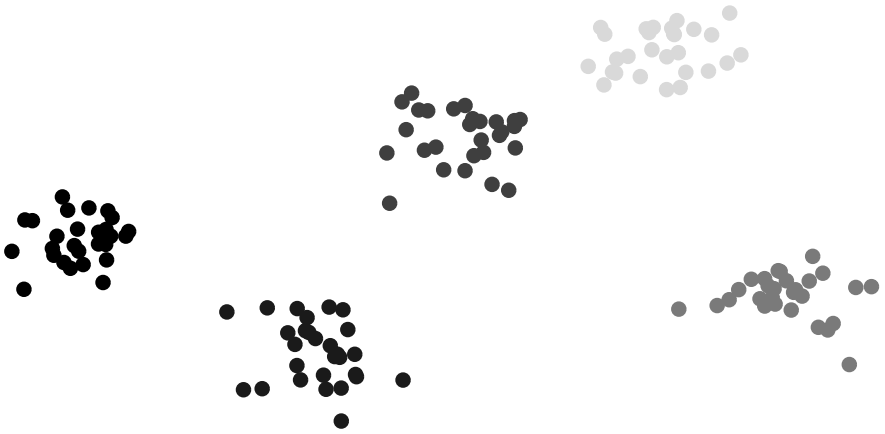


Fig 10. 2D UMAP projection of the embedding space (mock data).

5. Scheduling and Re-runs

Celery beat triggers the runner every hour by default. The runner can also be invoked manually from the admin dashboard or the CLI:

```
python -m pipeline.runner --once
python -m pipeline.runner --once --specialty radiology
```

Schedule	Trigger	Scope
Hourly	Celery beat	All specialties, incremental.
On demand	Admin button	One specialty, full re-cluster.
Nightly (option)	cron job	Full re-cluster, drop noise.

6. Performance Targets

Targets below are what we plan to hit on a 4 vCPU 8GB VM with 10,000 logged chats. Embedding dominates wall-clock time because it is the only stage that runs a neural model.

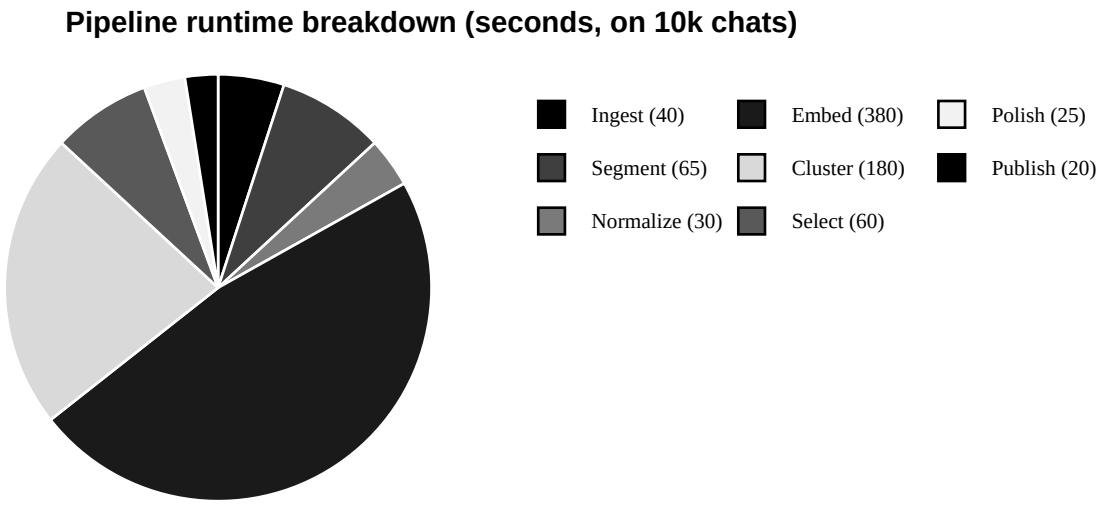


Fig 11. Time spent per stage on a 10k-chat run (target).

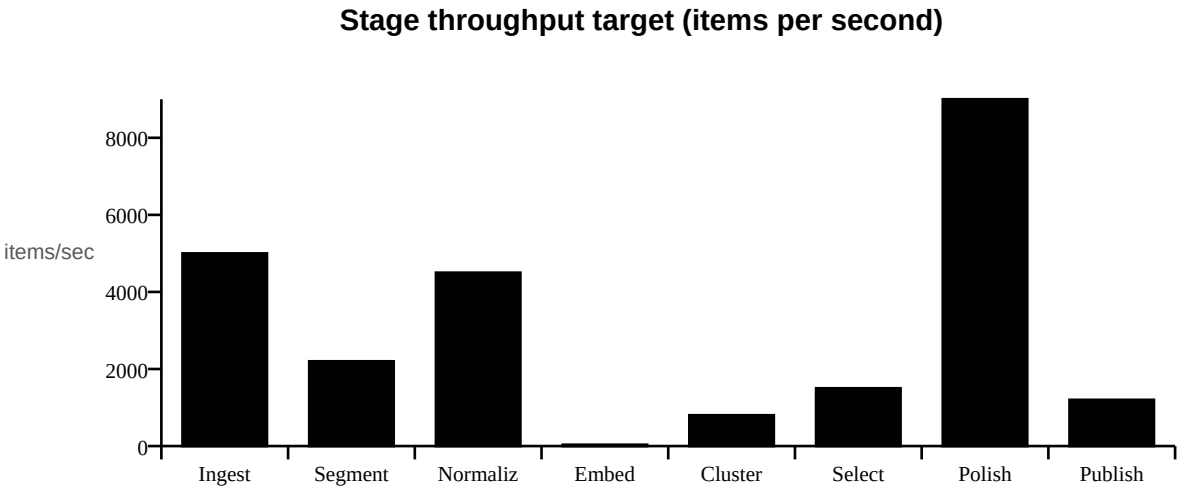


Fig 12. Per-stage throughput target (items/sec).

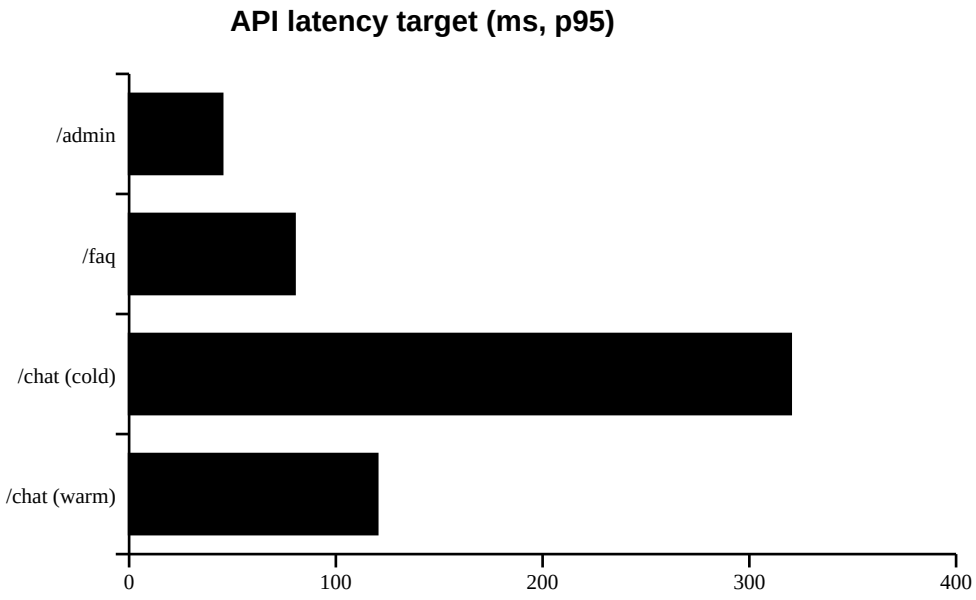


Fig 13. API latency target (p95, ms).

7. Quality Targets

Quality is measured at two places: cluster quality (intrinsic) and answer usefulness (extrinsic). Targets below are what we plan to report in the final paper.

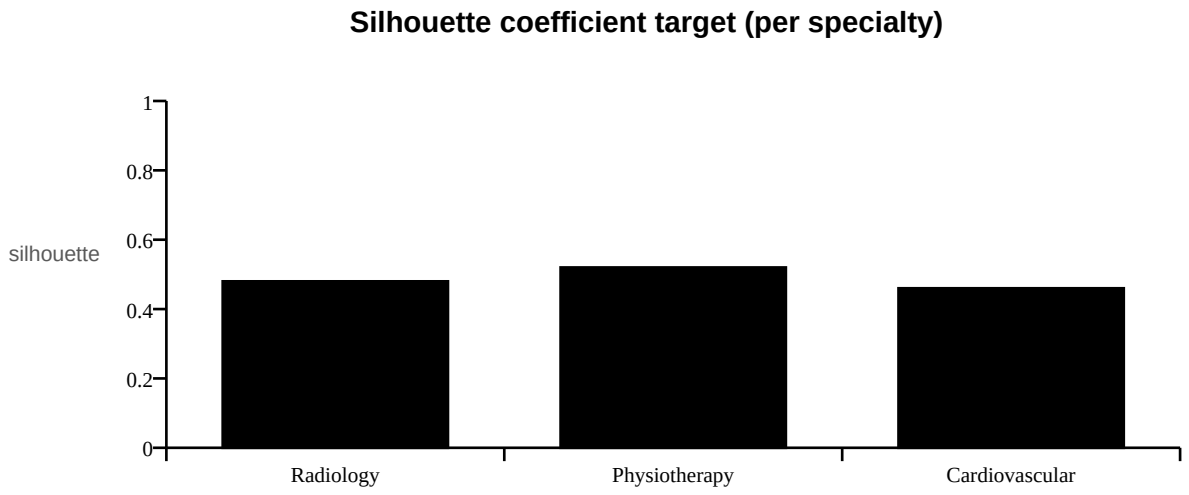


Fig 14. Silhouette coefficient target per specialty.

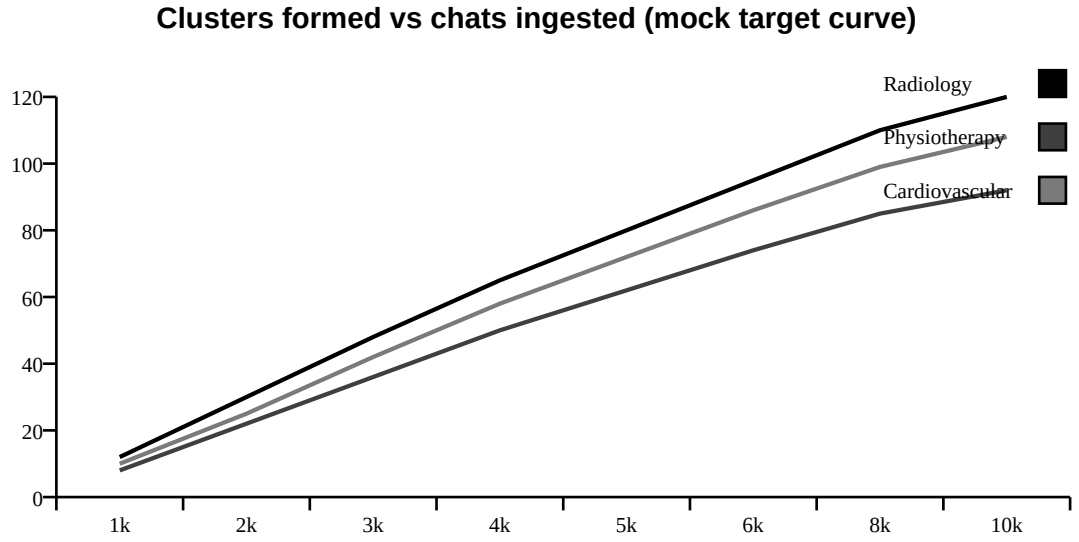


Fig 15. Cluster count growth as chats are ingested (target curve).